

Preemptive Scheduling Through Timer Interrupts

Trap Frame Inspection and the Timer-Driven Context-Switch Path in xv6

Author: Mishank Shah (24BCE2467)

Course: BCSE303P — Operating Systems Laboratory | **Base kernel:** xv6-public (MIT, x86)

Assignment: Lab DA-4 (VTOP Lab DA-5,6) — Preemptive Scheduling Through Timer Interrupts, due 1 October 2026

1. Objective

Every timer tick delivers IRQ0 to CPU 0 as a hardware interrupt, regardless of what that CPU happens to be executing at the time. By breaking on that interrupt inside GDB and inspecting the `struct trapframe` it produces, this lab traces, concretely, how xv6 forcibly takes the CPU away from whatever is running, preserves everything needed to resume it later, and uses that same mechanism

(`trap()` → `yield()` → `sched()` → `swtch()`) to give every runnable process a fair, unrequested turn at the CPU — the essential difference between *preemptive* and cooperative scheduling.

2. Setup

```
# terminal 1
make qemu-nox-gdb          # -s -S: gdbstub on tcp::26000, CPU halted at reset

# terminal 2
gdb
(gdb) file kernel
(gdb) target remote localhost:26000
```

Before attaching, `spin.c` — a trivial user program consisting of nothing but `while(1);` — was added to `UPROGS` and started from the shell (`$ spin &` -equivalent: run it, then switch consoles) so that at least one process is always busy, actively executing its own instructions, with nothing to make it trap voluntarily. This is what lets the interrupt hits in Part 2 land inside genuine user code rather than only ever catching a process blocked in a system call.

3. Part 1 — Trap Frame Inspection

A conditional breakpoint restricts `trap()` to firing only for the timer, out of the dozen or so reasons `trap()` is entered (syscalls, page faults, disk, keyboard, serial, and every other interrupt line):

```
(gdb) break trap if tf->trapno == T_IRQ0 + IRQ_TIMER
(gdb) continue
Continuing.

Breakpoint 1, trap (tf=0x8f9fffb4) at trap.c:41
41      switch(tf->trapno){
(gdb) print *tf
$1 = {edi = 0, esi = 0, ebp = 0, oesp = 0, ebx = 0, edx = 0, ecx = 0, eax = 0,
      gs = 0, padding1 = 0, fs = 0, padding2 = 0, es = 35, padding3 = 0,
      ds = 35, padding4 = 0, trapno = 32, err = 0,
      eip = 4160, cs = 27, padding5 = 0, eflags = 514,
      esp = 8172, ss = 35, padding6 = 0}
```

Field	Value	Meaning
trapno	32	<code>T_IRQ0 (32) + IRQ_TIMER (0)</code> — confirms this really is the timer, not another interrupt sharing the handler
eip	0x1040 (4160)	the next instruction the interrupted code would have executed — here, inside <code>spin</code> 's own <code>while(1);</code>
esp / ss	0x1FEC / 35 (0x23)	<code>spin</code> 's user-mode stack pointer and stack segment — present at all, because this trap crossed from ring 3 to ring 0
cs	27 (0x1B)	user code segment selector (<code>SEG_UCODE<<3 DPL_USER</code>) — proof the CPU was in user mode, not kernel mode, when the tick landed
eflags	514 (0x202)	bit 9 (<code>IF</code>) set — interrupts were enabled in the interrupted code, which is exactly why this interrupt was allowed to land

The struct itself (`x86.h`) explains why `esp / ss` are only meaningful here because a ring change occurred; had the timer instead interrupted kernel code already running in ring 0, those two fields would not be pushed at all:

```
struct trapframe {
    uint edi; uint esi; uint ebp; uint oesp;          // pushed by pusha (trapasm.S)
    uint ebx; uint edx; uint ecx; uint eax;

    ushort gs; ushort padding1;                      // segment regs, pushed explicitly
    ushort fs; ushort padding2;
    ushort es; ushort padding3;
    ushort ds; ushort padding4;
    uint trapno;                                       // which vector — pushed by the vector stub

    uint err;                                         // pushed by the CPU (0 if the vector has no e
    uint eip; ushort cs; ushort padding5; uint eflags; // pushed by the CPU, always
    uint esp; ushort ss; ushort padding6;           // pushed by the CPU, ONLY on a ring change
};
```

4. Part 2 — Three Timer-Interrupt Hits

`continue` three more times against the same conditional breakpoint, recording `tf->eip` and running `info symbol <eip>` on each:

Hit	eip	Code Running (from <code>info symbol</code>)
1	0x80106d81	<code>scheduler+45</code> in section <code>.text</code> — the tick landed inside the idle scan of <code>ptable</code> , before <code>spin</code> had been picked; <code>myproc()</code> / <code>c->proc</code> was <code>0</code> at this instant
2	0x00001040	No symbol matches <code>0x1040</code> . — inside <code>spin</code> 's own <code>while(1);</code> ; the address is real, but it is a <i>user-space</i> address and <code>file kernel</code> never loaded <code>spin</code> 's symbol table
3	0x80104a3f	<code>consolewrite+63</code> in section <code>.text</code> — <code>sh</code> was mid <code>write()</code> syscall (printing its prompt) when the tick fired; still kernel code, but running on <code>sh</code> 's behalf, not idling

All three eip values are real, resolvable addresses — the difference is only which world they belong to. Hits 1 and 3 are both "in the kernel", but one has no process attached at all and the other is the kernel acting as a process's agent; hit 2 is the only one that is genuinely outside the kernel binary altogether, which is exactly why the kernel's own symbol table has nothing to say about it.

5. Part 3 — Context Switch Path

```
(gdb) break sched
(gdb) continue
Continuing.

Breakpoint 2, sched () at proc.c:341
341      int intena;
(gdb) backtrace
#0  sched () at proc.c:341
#1  0x80104f2c in yield () at proc.c:361
#2  0x8010328e in trap (tf=0x8f9fffb4) at trap.c:53
#3  0x00107a10 in alltraps ()
#4  0x00001040 in ?? ()
(gdb) print *p
$1 = {sz = 12288, pgdir = 0x8f9d6000, kstack = 0x8f9ff000 "",
      state = RUNNABLE, pid = 7, parent = 0x8f9fe4a0, tf = 0x8f9fffb4,
      context = 0x8f9fff9c, chan = 0x0, killed = 0, name = "spin"}
(gdb) print p->tf->eip
$2 = 4160
```

The backtrace confirms the exact chain implied by `trap.c`'s own final check (`if(myproc() && myproc()->state == RUNNING && tf->trapno == T_IRQ0+IRQ_TIMER) yield();`): the interrupt vector stub (`alltraps`) called `trap()`, which called `yield()`, which called `sched()`. By the time `sched()`'s breakpoint is hit, `yield()` has already flipped this process's state — `p->state == RUNNABLE`, not `RUNNING` — which is itself required by `sched()`'s own assertion (`if(p->state == RUNNING) panic("sched running");`). `p->tf->eip` is `4160`, exactly the value recorded for Hit 2 in Part 2: this is the same trapframe, on the same kernel stack, simply inspected one call frame deeper.

In this xv6-public tree, the current process is reached through `myproc()` (equivalently `mycpu()->proc`) rather than a single global `proc` pointer, and inside `sched()` it is already bound to the local `p` (`struct proc *p = myproc();`) — so `print *p` is the direct equivalent of the worksheet's `print *proc`. That local can never actually be `NULL` at this particular breakpoint, since `sched()` is only ever reached from `yield()` / `sleep()` / `exit()`, all of which require a real running process to call them. A transient `NULL` (as Hit 1 in Part 2 shows) is only visible one level up, inside `scheduler()`'s own scan, where `c->proc` is explicitly set to `0` between processes.

6. Observations and Explanations

a. Why does the saved eip differ on each hit? What does this say about when preemption can occur relative to a process's own code?

The timer is driven by the LAPIC's own countdown, entirely independent of whatever instruction stream happens to be executing when it expires — so `eip` is simply "whatever was about to run" at an essentially arbitrary moment. The three hits landed in three different regimes: idle kernel code with no process attached (`scheduler()`'s scan), a process's own user-space instructions (`spin`'s loop), and kernel code executing on a process's behalf inside a system call (`consolewrite`). That spread is the point: preemption is not scheduled around a process's code at some cooperative checkpoint it chooses (a loop boundary, a syscall return) — it can interrupt literally any instruction boundary at which interrupts happen to be enabled, which is what makes it *preemptive* rather than voluntary.

b. What would happen without timer interrupts — specifically to a process running `while(1);`?

The only path back into the scheduler on a timer tick is the check at the end of `trap()`: `if(myproc() && myproc()->state==RUNNING && tf->trapno==T_IRQ0+IRQ_TIMER) yield();`. With no timer interrupt, that condition is never even evaluated for that reason — the only other way back into `trap()` is a syscall or a fault, and `while(1);` makes neither: it never touches memory it doesn't own and never calls into the kernel. Once `spin` starts running, the CPU running it never traps again, never calls `yield()`, and `scheduler()` on that CPU never regains control. Every other process — including `init` and the shell — simply never runs again on that CPU. This is precisely the failure mode timer-driven preemption exists to rule out.

c. How do the saved eip/esp/eflags in the trap frame make it possible to resume the exact same instruction stream later?

Taking the interrupt is itself what creates the record: the CPU hardware pushes `eip`, `cs`, `eflags` (and `esp` / `ss`, since this trap crossed rings) onto the interrupted process's own kernel stack before `trap()` ever runs; `alltraps` (`trapasm.S`) then adds the general-purpose and segment registers with `pusha`, completing the `struct trapframe` that `p->tf` points at. `eip` is the exact next instruction, `esp` its exact stack pointer, and `eflags` its exact condition-code and interrupt-enable state — together a complete snapshot of everything about to be lost. Resuming is the same transaction in reverse: `trapret` pops the general-purpose and segment registers back off that same trapframe and executes `iret`, which reloads `eip`, `cs`, `eflags`, `esp`, and `ss` atomically. The CPU has no way to distinguish "this instruction stream ran uninterrupted" from "this instruction stream was suspended for however long the kernel spent running other processes" — which is exactly the illusion preemptive multitasking depends on.

7. Conclusion

Breaking on `trap()` conditioned on the timer vector, and following the resulting trapframe through `yield()` and into `sched()`, made the entire preemption mechanism observable rather than theoretical: a hardware interrupt that can land inside idle kernel code, a process's own instructions, or kernel code running on a process's behalf, indiscriminately; a `struct trapframe` that captures everything needed to resume whatever was interrupted, regardless of which of those three regimes it was in; and a fixed call chain (`trap → yield → sched → swtch`) that turns that one interrupt into a fair, involuntary handoff of the CPU to the next `RUNNABLE` process. Without it, as Part 6(b) shows, a single `while(1);` would be enough to freeze a CPU core permanently.